

COMPILER CONSTRUCTION LAB

PROJECT: FOOD DELIVERY INTERPRETER



Sr #	Name	Roll no	Section
1	Sadia Asif	Fa23/BSCS/357	I
2	Komal Yousaf	Fa23/BSCS/334	I
3	Mahmoona khan	Fa23/BSCS/336	I
4	Sumamah	Fa23/BSCS/686	I

Supervised By:

Azib Mehmood: _____
(Signature)

**DEPARTMENT OF COMPUTER SCIENCE
LAHORE GARRISON UNIVERSITY**

Abstract

This project implements a Mini-Compiler for an Automated Catering System using a custom Domain-Specific Language (DSL). The system processes simple food-ordering commands and translates them through a complete 7-phase compiler pipeline including lexical analysis, syntax analysis, semantic checking, parse tree generation, intermediate code generation, optimization, and target code generation.

Built using Flex and Bison, the compiler validates user commands, prevents invalid operations, and converts high-level instructions into simulated low-level assembly code. The project follows a hybrid single-pass architecture, combining interpreter-like interaction with compiler-based backend processing.

The main goal of this project is to demonstrate compiler design concepts in a practical and easy-to-understand real-world scenario while highlighting the importance of DSLs in automation, embedded systems, and smart industrial applications.

1. Executive Summary & Project Philosophy

1.1 Introduction to the Mini-Compiler Project

Traditional software applications rely on heavy, general-purpose frameworks to process simple tasks. This project takes a different approach by treating a restaurant ordering system as a language translation problem. By defining a customized language specification for food ordering, this project implements a fully realized, 7-phase mini-compiler pipeline. The system reads human-readable text commands, analyzes them for correctness, and translates them into simulated machine-level instructions.

1.2 Introduction to Domain-Specific Languages (DSL)

A Domain-Specific Language (DSL) is a programming or command language built for one specific job. General languages like Python or C++ can do everything, but they require a lot of setup and complex code. A DSL cuts away the clutter. In this project, the DSL is designed purely for an automated catering environment. It allows users or kitchen staff to type simple commands (like `ORDER pizza 2`) and let the compiler handle the heavy lifting safely and efficiently.

1.3 Architectural Identity: Hybrid Single-Pass Compiler vs. Interpreter

If someone asks whether this is a compiler or an interpreter, the answer is that it is a **single-pass hybrid**.

- **Interpreter-like behavior:** It takes input line-by-line in a terminal shell and immediately updates the system (like adding items to a cart).
- **Compiler-like behavior:** Before it executes anything, the system strictly pauses to pass the text through the back-end phases of a classic compiler - generating intermediate code, running an optimization check, and outputting target assembly code.

1.4 Scope and Objectives of the Automated Catering Compiler

The main objective of this system is to demonstrate how a full compiler pipeline works in a relatable scenario.

- **Scope:** The project is bounded by a specific grammar (ordering, removing items, calculating totals).
- **Target Users:** Lab instructors looking for a complete pipeline demonstration, or simulated restaurant staff.
- **Design Goals:** Fast tokenization, strict logical safety (no negative orders), and clear visibility into how high-level text becomes low-level machine code.

2. Complete Compiler Pipeline Architectural Blueprint

2.1 The 7-Phase Core Mapping Matrix

Here is exactly how the traditional academic compiler phases map to our project:

Phase	Academic Name	Our Project Module	What It Does
1	Lexical Analysis	Flex Scanner (food.l)	Chops text into recognized "tokens" (Keywords, Numbers, IDs).
2	Syntax Analysis	Bison Parser (food.y)	Checks if the tokens are arranged in a valid grammatical order.
3	Semantic Analysis	semantic_check()	Checks if the command makes logical sense (e.g., does the food exist?).
4	Parse Tree Gen.	print_tree()	Draws a visual map of how the grammar matched the sentence.
5	Intermediate Code	generate_icg()	Translates the command into platform-independent bytecode.
6	Optimization	optimize_code()	Checks for ways to make the execution faster or use less memory.
7	Target Code	generate_target_code()	Writes simulated hardware assembly instructions (e.g., MOV R1).

2.2 End-to-End Statement Trace Layout (ORDER pizza 2)

When you type ORDER pizza 2, here is its exact journey:

1. **Flex** reads the text, ignores the spaces, and extracts the tokens: ORDER, "pizza", and 2.
2. **Bison** sees these tokens and matches them to the rule ORDER ID NUMBER.
3. The **Semantic Analyzer** checks the menu database to confirm "pizza" is real and the quantity is above zero.

4. The **Parse Tree** function prints a visual branch on the screen.
5. The **Intermediate Code** generator prints `ADD_TO_CART pizza 2`.
6. The **Optimizer** logs that it has streamlined the memory allocation for this item.
7. The **Target Code** generator outputs hardware instructions: `MOV R1, #2; STR R1, [pizza_PTR]`.

2.3 Communication Mechanisms: How Lexer and Parser Interact

The Lexer (Flex) and Parser (Bison) are separate tools, so they need a bridge to communicate. Bison generates a header file (`food.tab.h`) that Flex includes.

When Flex finds a value (like the number 2 or the word "pizza"), it stores it in a special global variable called `yylval`. Bison then picks up that value from `yylval` to use in its grammar rules.

3. Phase Functional Deep-Dive

3.1 Frontend: The Analysis Phase

3.1.1 Phase 1: Lexical Analysis (Flex Scanner Engine)

The Lexer is the reader. It uses regular expressions (regex) to scan incoming text. It ignores trivial things like whitespace and tabs. If it sees "ORDER", it tags it as a command. If it sees numbers, it converts them into integers.

3.1.2 Phase 2: Syntax Analysis (Bison Grammar Specification)

The syntax phase acts as the grammar police. It uses Context-Free Grammar (CFG) rules. Even if the words are spelled correctly, they must be in the right order. For example, `ORDER 2 pizza` will fail because the grammar strictly expects `ORDER [Name] [Quantity]`.

3.1.3 Phase 3: Semantic Analysis (Contextual and Database Verification)

Syntax checks structure; Semantics checks meaning. You can grammatically say `ORDER unicorn 50`, but the semantic phase will block it because "unicorn" is not in the database. It also blocks out-of-bounds requests, like ordering -5 burgers.

3.1.4 Phase 4: Parse Tree Generation (Visual Structural Logging)

This phase takes the matched grammar and logs a visual structural tree to the console. It helps the user understand exactly how the compiler interpreted their sentence hierarchy.

3.2 Backend: The Synthesis Phase

3.2.1 Phase 5: Intermediate Code Generation (Platform-Neutral IR)

Before translating to specific hardware code, a good compiler translates to an Intermediate Representation (IR). This is a clean, neutral instruction format (like Java Bytecode) that isn't tied to an Intel or ARM processor.

3.2.2 Phase 6: Localized Code Optimization (Redundancy Elimination)

This phase inspects the intermediate code for inefficiencies. While our project prints simulated optimization logs for educational purposes, in a real system, this phase would collapse redundant variables, inline constants, and clear up memory bottlenecks.

3.2.3 Phase 7: Target Code Generation (Pseudo-Assembly Register Emission)

The final stage maps the operations directly to simulated hardware components. It translates the human intent into low-level CPU instructions, utilizing mock registers (R1, ACC) and pointers.

4. Core Production Code Implementations

4.1 Memory Allocation & Shared Definitions Bridge

Because we are passing dynamic strings (like food names) from the Lexer to the Parser, we must manage memory carefully. In the Lexer, when a food name is found, it is copied into memory using `strdup()`. Once the Parser finishes processing that command, it immediately calls `free()` to prevent memory leaks.

4.2 Frontend Tokenization Specification Configuration (food.l)

This file handles the pattern matching.

Code snippet

```
"MENU"          { return MENU; }
"ORDER"         { return ORDER; }
[0-9]+          { yylval.num = atoi(yytext); return NUMBER; }
[a-zA-Z_][a-zA-Z0-9_]* { yylval.str = strdup(yytext); return ID; }
[ \t]+          { /* Ignore whitespace */ }
```

4.3 Backend Production Rules and Pipeline Orchestration (food.y)

This file contains the pipeline glue. Here is how a command triggers the backend:

C

command:

```
ORDER ID NUMBER {
    if(semantic_check($2, $3)) {
        generate_icg("ORDER", $2, $3);
        optimize_code("ORDER");
        generate_target_code("ORDER", $2, $3);
        // Execute runtime state updates
    }
    free($2);
}
```

5. Pipeline Execution Trace & Validation Matrix

5.1 Compilation and Toolchain Build Guide

To run this project, the following terminal commands are required:

1. `bison -d food.y` (Generates the parser logic and header file)
2. `flex food.l` (Generates the lexer C code)
3. `gcc food.tab.c lex.yy.c -o food` (Compiles both into a single executable)
4. `./food` (Runs the compiler shell)

5.2 Test Case Scenarios and Valid Terminal Run Logs

Input: ORDER pizza 2

Output Log:

Plaintext

[PARSE TREE]

```
statement
├── order_stmt
│   └── ITEM: pizza | QTY: 2
```

[ICG] ADD_TO_CART pizza 2

[OPTIMIZER] Optimized repeated memory allocations for cart items.

[TARGET CODE] MOV R1, #2; STR R1, [pizza_PTR]

[RUNTIME OK] Added 2x pizza to your cart.

5.3 Error Recovery Verification Matrix

The system is built to safely recover from bad input:

- **Lexical Error:** Input ORDER @pizza. The lexer flags the @ as an unknown character and drops it, rather than crashing.
- **Syntax Error:** Input ORDER 2. The parser rejects it for missing the item ID.
- **Semantic Error:** Input ORDER shoes 2. The semantic checker throws a database error and halts the backend phases.

6. Analytical Evaluation: Gaps & Architectural Scaling

6.1 Flat Mock Registers vs. Dynamic Symbol Table Chaining

Our project uses a flat global array (`cart[]`) to act as our variable memory. In a large-scale commercial compiler, this is replaced by a "Symbol Table"—a complex hash map that tracks variable scope, memory addresses, and data types across different functions.

6.2 Line-by-Line Streaming Blocks vs. In-Memory Abstract Syntax Trees (AST)

This compiler processes code one line at a time. Enterprise compilers parse the entire source file and build a massive tree data structure in memory called an AST. This allows them to analyze global flow before generating a single line of machine code.

6.3 Virtual Machine Console Outputs vs. External Binary Payloads

Our target code generation prints assembly instructions to the terminal console. A true production compiler writes those binary instructions into an actual executable file (like an .exe) saved on a hard drive to be run by the operating system later.

7. Conclusion, Demand, Importance, Benefits & Future Enhancements

7.1 Project Retrospective Summary

This project successfully demystifies the "black box" of compiler design. By stepping away from compiling heavy programming languages and focusing on a Domain-Specific Language (DSL) for catering, the project clearly illustrates how high-level human intent is mathematically translated into safe, optimized machine instructions.

7.2 The Demand for This Project Pattern

In modern industry, there is a massive and growing demand for Domain-Specific Languages.

- **Industrial Automation:** Warehouses, smart kitchens, and manufacturing plants do not want their floor staff writing complex database code. They demand restricted, safe command languages.
- **IoT and Edge Computing:** Smart appliances require tiny, lightweight translation engines. Because this project relies on a single-pass Flex/Bison setup, it can run on very weak hardware (like a microcontroller inside an oven or kiosk) without the heavy overhead of web-based applications.

7.3 Importance and Core Benefits

The benefits of implementing a software solution as a compiler rather than a standard script include:

- **Proactive Semantic Safety:** Erroneous or malicious inputs (like ordering a negative amount of food) are mathematically trapped and destroyed *before* they ever reach the backend database.
- **Total Architectural Separation:** Because the front-end (parsing) and back-end (code generation) are separated, if the restaurant buys new hardware, the development team only needs to update the Target Code phase. The rest of the software remains untouched.
- **Zero Bloat:** It processes text incredibly fast, taking up almost zero system memory.

7.4 Roadmap for Industrial Scaling & Future Improvements

To take this project from an academic lab to a commercial enterprise product, the following future improvements are required:

1. **Multi-Pass AST Architecture:** Upgrade the parser to read entire order script files (batch processing) and build a full Abstract Syntax Tree in memory to analyze global errors before executing.
2. **Dynamic Symbol Tables:** Replace the flat arrays with dynamic Hash Tables to support multi-user sessions, table-specific variables, and promotional discount scoping.
3. **Real Data-Flow Optimization:** Enhance the optimizer to perform actual "Constant Folding" (automatically turning ORDER pizza 1+1 into ORDER pizza 2) and redundancy elimination.
4. **Physical File Output:** Upgrade the Target Code Generator to write actual .asm or binary bytecode files to the disk, which could be independently executed by kitchen display hardware.